In the present document, I summarize the schemes of the programs.

1) The general structure of the program for constructing the energy distribution functions.
2) The expression for the pair energy and an efficient scheme for insertion in the Ewald method.
3) The reciprocal part of Ewald and particle-mesh Ewald methods.
4) The functional for the solvation free energy when the energy coordinate is discretized.
5) The self-energy appearing in Ewald and PME methods.
6) The “standard” mesh size of energy coordinate
7) The scheme to read a trajectory, obtain the energy distribution functions using ermod program, and calculate the solvation free energy
8) semi-automatic flow of calculations

1) General structure of the program for constructing the energy distribution functions

The structure of enganal.f (my main program for constructing the energy distribution functions) is as follows;

```
Initialization: ρ = 0 ; ρ0 = 0 ; n(2) = 0

do MD_count=1, MD_MAX    (MD_MAX: # of snapshots examined )
    if solution    X= 1
    if insertion    X= (X: # of insertions)
    do num_cnf=1,X
        if solution    read_configuration(solute,solvent)
        if insertion    then
            read_configuration(solvent)
            call insertion (solute)
        endif
        hist=0                              ! initialization of instantaneous histogram
        do i=1,(# of solvent molecules)
          get_pair_energy_between_solute_and_solvent_i
          determine_the_mesh_number_j_corresponding_to_the_pair_energy
          hist(j)= hist(j)+1
        enddo
        if solution    ρ = ρ + hist
        if insertion    then
          ρ0= ρ0 + hist
          do i over the meshes ; do j over the meshes
```

$$n^{(2)}(i,j) = n^{(2)}(i,j) + \mathrm{hist}(i) * \mathrm{hist}(j)$$

```
          enddo ; enddo
        endif
    enddo
enddo
if solution    ρ = ρ/X/MD_MAX
if insertion    ρ0 = ρ0/X/MD_MAX ;    n(2) = n(2)/X/MD_MAX

Save_the_distribution_functions
```

2) The expression for the pair energy and insertion tip in the Ewald method

Let α denote the site of the solute and β denote the site of the jth solvent molecule. The pair interaction energy in the (full) Ewald scheme between the solute and the jth solvent molecule is given by

$$\sum_{\alpha}\sum_{\beta}\left[4\varepsilon_{\alpha\beta}\left\{\left(\frac{\sigma_{\alpha\beta}}{r_{\alpha,j\beta}}\right)^{12}-\left(\frac{\sigma_{\alpha\beta}}{r_{\alpha,j\beta}}\right)^{6}\right\}+Q_{\alpha}q_{j\beta}\frac{\operatorname{erfc}\left(\kappa r_{\alpha,j\beta}\right)}{r_{\alpha,j\beta}}\right]+\sum_{\alpha}\sum_{\beta}\sum_{\mathbf{k}}Q_{\alpha}q_{j\beta}\frac{4\pi}{V}\frac{\exp\left(-|\mathbf{k}|^{2}/4\kappa^{2}\right)}{|\mathbf{k}|^{2}}\cos\left(i\mathbf{k}\cdot\left(\mathbf{r}_{\alpha}-\mathbf{r}_{j\beta}\right)\right)-\frac{\pi}{\kappa^{2}V}\left(\sum_{\alpha}Q_{\alpha}\right)\left(\sum_{\beta}q_{j\beta}\right)$$

$$=\sum_{\alpha}\sum_{\beta}\left[4\varepsilon_{\alpha\beta}\left\{\left(\frac{\sigma_{\alpha\beta}}{r_{\alpha,j\beta}}\right)^{12}-\left(\frac{\sigma_{\alpha\beta}}{r_{\alpha,j\beta}}\right)^{6}\right\}+Q_{\alpha}q_{j\beta}\frac{\operatorname{erfc}\left(\kappa r_{\alpha,j\beta}\right)}{r_{\alpha,j\beta}}\right]+\sum_{\alpha}\sum_{\beta}\sum_{\mathbf{k}}Q_{\alpha}q_{j\beta}\frac{4\pi}{V}\frac{\exp\left(-|\mathbf{k}|^{2}/4\kappa^{2}\right)}{|\mathbf{k}|^{2}}\operatorname{Re}\left(\exp\left(i\mathbf{k}\cdot\left(\mathbf{r}_{\alpha}-\mathbf{r}_{j\beta}\right)\right)\right)-\frac{\pi}{\kappa^{2}V}\left(\sum_{\alpha}Q_{\alpha}\right)\left(\sum_{\beta}q_{j\beta}\right)$$

where

$\mathbf{r}_{\alpha}$ is the position of the site α of the solute,

$\mathbf{r}_{j\beta}$ is the position of the site β of the jth solvent molecule,

$r_{\alpha j\beta}$ is the distance between the site α of the solute and the site β of the jth solvent expressed as $r_{\alpha,j\beta}=\left|\mathbf{r}_{\alpha}-\mathbf{r}_{j\beta}\right|$

Q_{α} is the partial charge of the site α of the solute

$q_{j\beta}$ is the partial charge of the site β of the jth solvent molecule

erfc is the complementary error function

$\varepsilon_{\alpha\beta}$ and $\sigma_{\alpha\beta}$ are the LJ parameters between the site α of the solute and the site β of the jth solvent which is typically constructed by the Lorentz-Berthelot rule

V is the system volume

$\mathbf{k}$ is the reciprocal vector given by $\mathbf{k} = (2\pi/L)\mathbf{n}$, with the cell size L and integer vector $\mathbf{n}$

κ is the screening constant for the Ewald sum

To convert to the SI unit, the real part of the Coulomb energy is multiplied by $1/4\pi\varepsilon_0$ and in the reciprocal part, $4\pi/V$ is changed into $1/V\varepsilon_0$.

For the LJ and real-space part of electrostatic interaction, one needs to calculate the above expression for each insertion. When he does the solute insertions many times for a single reference solvent configuration, it is possible to speedup the calculations of the reciprocal part.

Let the solvent be the 1 to N^{th} molecules. Further, as another notation,

$$Q(\mathbf{k}) = \sum_{\alpha} Q_{\alpha} \exp(i\mathbf{k}\cdot\mathbf{r}_{\alpha})$$
$$q_j(\mathbf{k}) = \sum_{\beta} q_{j\beta} \exp(i\mathbf{k}\cdot\mathbf{r}_{j\beta})$$

And the asterisk (*) in the following means the complex conjugate.

At any instantaneous configuration of the reference solvent,

a) Calculate $q_i(\mathbf{k})$ for the solvent

(The set of $q_i(\mathbf{k})$ is common to all the solute insertions performed at the present instantaneous configuration of the reference solvent)

b) do ins=1,M (ins counts the solute insertion, and M is its maximum)

b1) Insert the solute randomly with random orientation

The solute conformation is sampled according to the pre-constructed distribution at isolated state

b2) Calculate $Q(\mathbf{k})$ for the solute

b3) do j=1,N

Calculate
$$\sum_{\alpha}\sum_{\beta}\left[4\varepsilon_{\alpha\beta}\left\{\left(\frac{\sigma_{\alpha\beta}}{r_{\alpha,j\beta}}\right)^{12}-\left(\frac{\sigma_{\alpha\beta}}{r_{\alpha,j\beta}}\right)^{6}\right\}+Q_{\alpha}q_{j\beta}\frac{\mathrm{erfc}(\kappa r_{\alpha,j\beta})}{r_{\alpha,j\beta}}\right]$$
$$+\sum_{\mathbf{k}}\mathrm{Re}\left(Q(\mathbf{k})[q_j(\mathbf{k})]^*\right)\frac{4\pi}{V}\frac{\exp\left(-|\mathbf{k}|^2/4\kappa^2\right)}{|\mathbf{k}|^2}-\frac{\pi}{\kappa^2 V}\left(\sum_{\alpha}Q_{\alpha}\right)\left(\sum_{\beta}q_{j\beta}\right)$$

(The asterisk (*) means the complex conjugate)

(This is the pair interaction energy between the solute and j-th solvent)

enddo

b4) save the array of the solute-solvent pair interaction energy

enddo

If the number of **k** vectors is K, and if one calculates the reciprocal part without using the above method, the computation for the reciprocal part is performed (KNM) times at each instantaneous configuration of the reference solvent with maxins times of solute insertions.

When the above procedure is followed, the step a) takes KN computations and is done only once for an instantaneous configuration of the reference solvent. The step b) then takes (NM) computations for the calculation of the reciprocal part. Thus, at each instantaneous configuration of the reference solvent, the computation for the reciprocal part is ($KN + NM$) times for with maxins times of solute insertions. Of course, for large K, N, and M, $KN + NM << KNM$

3) Reciprocal part in Ewald and PME methods

* See 5) for the effect of the self-energy on the solvation free energy.

I mostly follow Essmann, Perera, Berkowitz, Darden, Lee, and Pedersen (*J. Chem. Phys.* **103,** 8577, 1995) method of smooth particle mesh Ewald.

The unit cell is rectangular with the edge lengths of L_x, L_y, and L_z.
The reciprocal lattice vector $\mathbf{m} = (m_x/L_x, m_y/L_y, m_z/L_z)$ with integers (m_x, m_y, m_z).

The structure factor $S(m_x,m_y,m_z;i)$ for the i-th molecule is given by

$$S(m_x, m_y, m_z; i) = \sum_{\alpha} q_{i\alpha} \exp(-2\pi i \mathbf{m} \cdot \mathbf{r}_{i\alpha})$$

where α is the site in the i-th particle, $q_{i\alpha}$ is the charge on the α-th site of the i-th particle, and $\mathbf{r}_{i\alpha}$ is the position of the α-th site of the i-th particle.
The sign convention in the exponential factor is opposite to that employed by Essmann *et al*, but it does not affect any of the following development.
My sign convention follows the one used in the discrete Fourier transform routine of Intel Math Kernel Library, which I use for 3D-FFT.

In the full Ewald method, the electrostatic part of the pair energy between the i-th and j-th particles ($i \neq j$) is

$$\sum_{\alpha,\gamma} q_{i\alpha} q_{j\gamma} \frac{\mathrm{erfc}(\kappa r_{i\alpha.j\gamma})}{r_{i\alpha.j\gamma}} + \frac{1}{\pi V} \sum_{(m_x,m_y,m_z)\neq 0} \frac{\exp(-\pi^2 \mathbf{m}^2/\kappa^2)}{\mathbf{m}^2} \mathrm{Re}\left(S(m_x, m_y, m_z; i) S^*(m_x, m_y, m_z; j)\right) - \frac{\pi}{\kappa^2 V} \left(\sum_{\alpha} q_{i\alpha}\right)\left(\sum_{\beta} q_{j\beta}\right)$$

where γ denotes the site in the j-th particle, κ is the screening factor, V is the system volume, S^* is the complex conjugate, and the $r_{i\alpha,j\gamma}$ is the minimum-image distance between the α-th site of the i-th particle and the γ-th site of the j-th particle.
The electrostatic part of the self-energy of the i-th particle is

$$\sum_{(\alpha,\gamma)\notin M} q_{i\alpha} q_{j\gamma} \frac{\mathrm{erfc}(\kappa r_{i\alpha.i\gamma})}{r_{i\alpha.i\gamma}} - \sum_{(\alpha,\gamma)\in M} q_{i\alpha} q_{j\gamma} \frac{\mathrm{erf}(\kappa r_{i\alpha.i\gamma})}{r_{i\alpha.i\gamma}} - \frac{\kappa}{\sqrt{\pi}} \sum_{\alpha} q_{i\alpha}{}^2$$
$$+ \frac{1}{2\pi V} \sum_{(m_x,m_y,m_z)\neq 0} \frac{\exp(-\pi^2 \mathbf{m}^2/\kappa^2)}{\mathbf{m}^2} \left|S(m_x, m_y, m_z; i)\right|^2 - \frac{\pi}{2\kappa^2 V} \left(\sum_{\alpha} q_{i\alpha}\right)^2$$

where the sum in the first term is taken over the distinct pairs (α,γ) of sites not belonging to the masked pairlist M and the sum in the second term is over those belonging to M.

Now I proceed to the PME.

The mesh size is K_x, K_y, and K_z along the x, y, and z directions, respectively.

The scaled fractional coordinate for a position (x, y, z) is given by

$$(u_x, u_y, u_z) = (xK_x/L_x, yK_y/L_y, zK_z/L_z).$$

The grid point in the space is specified by integers (k_x, k_y, k_z) with

$$0 \leqq k_x \leqq K_x - 1,\ 0 \leqq k_y \leqq K_y - 1,\ 0 \leqq k_z \leqq K_z - 1.$$

The spline function is denoted by $M_n(s)$ and is given by

$$M_n(s) = \frac{1}{(n-1)!}\sum_{i=0}^{n}(-1)^i \frac{n!}{i!(n-i)!}\left[\max(s-i,0)\right]^{n-1}$$
$$= \sum_{i=0}^{[s]}(-1)^i \frac{n}{i!(n-i)!}(s-i)^{n-1}$$

The array $Q(k_x, k_y, k_z;i)$ of the grid charge for the i-th particle is given by

$$Q(k_x, k_y, k_z; i)$$
$$= \sum_{\alpha}\sum_{l_x,l_y,l_z} q_{i\alpha} M_n(u_{ix} - k_x - l_x K_x) M_n(u_{iy} - k_y - l_x K_y) M_n(u_{iz} - k_z - l_z K_z)$$

Actually, since $M_n(s)$ is nonzero only when $0 < s < n$,

$M_n(u\text{-}k\text{-}lK)$ is nonzero only when $k+lK = [u], [u]\text{-}1, \ldots, [u]\text{-}n+1$

Thus, the above is rewritten as

$$Q(k_x, k_y, k_z; i)$$
$$= \sum_{\alpha} q_{i\alpha} \sum_{l_x=0}^{n-1}\sum_{l_y=0}^{n-1}\sum_{l_z=0}^{n-1} M_n(u_{ix} - [u_{ix}] + l_x) M_n(u_{iy} - [u_{iy}] + l_y) M_n(u_{iz} - [u_{iz}] + l_z)$$

and k is set to k = modulo([u]-l, K) (l = 0, … , n-1).

The Fourier transform $F(m_x, m_y, m_z; i)$ of the array $Q(k_x, k_y, k_z; i)$ is expressed as

$$F(m_x, m_y, m_z; i)$$
$$= \sum_{k_x=0}^{K_x-1}\sum_{k_y=0}^{K_y-1}\sum_{k_z=0}^{K_z-1} Q(k_x, k_y, k_z; i) \exp\left[-2\pi i\left(\frac{m_x k_x}{K_x} + \frac{m_y k_y}{K_y} + \frac{m_z k_z}{K_z}\right)\right]$$

(m_x, m_y, m_z) is integer with $0 \leqq m_x \leqq K_x - 1$, $0 \leqq m_y \leqq K_y - 1$, $0 \leqq m_z \leqq K_z - 1$

The sign convention in the exponential factor is opposite to that employed by Essmann

et al, but it does not affect any of the following development.
My sign convention follows the one used in the discrete Fourier transform routine of Intel Math Kernel Library.

Further, I define a function D as

$$D(m_x, m_y, m_z) = \frac{1}{\pi V} \frac{\exp\left(-\pi^2 \mathbf{m}^2 / \kappa^2\right)}{\mathbf{m}^2} \prod_{j=x,y,z} \frac{1}{\left|\sum_{h=0}^{n-2} M_n(h+1) \exp\left(-2\pi i h m_j / K_j\right)\right|^2}$$

where $\mathbf{m} = (m_x'/L_x, m_y'/L_y, m_z'/L_z)$ with
$m' = m$ when $m \leqq K/2$ and $m' = m - K$ when $m > K/2$
and $D = 0$ at $m_x = m_y = m_z = 0$

The Coulombic part of the pair energy between the i-th and j-th particles ($i \neq j$) is

$$\sum_{\alpha,\gamma} q_{i\alpha} q_{j\gamma} \frac{\mathrm{erfc}\left(\kappa r_{i\alpha.j\gamma}\right)}{r_{i\alpha.j\gamma}} + \sum_{m_x,m_y,m_z} D(m_x, m_y, m_z) F(m_x, m_y, m_z; i) F^*(m_x, m_y, m_z; j) - \frac{\pi}{\kappa^2 V} \left(\sum_{\alpha} q_{i\alpha}\right)\left(\sum_{\beta} q_{j\beta}\right)$$

The self-energy of the i-th particle is

$$\sum_{(\alpha,\gamma) \notin M} q_{i\alpha} q_{j\gamma} \frac{\mathrm{erfc}\left(\kappa r_{i\alpha.i\gamma}\right)}{r_{i\alpha.i\gamma}} - \sum_{(\alpha,\gamma) \in M} q_{i\alpha} q_{j\gamma} \frac{\mathrm{erf}\left(\kappa r_{i\alpha.i\gamma}\right)}{r_{i\alpha.i\gamma}} - \frac{\kappa}{\sqrt{\pi}} \sum_{\alpha} q_{i\alpha}{}^2$$
$$+ \frac{1}{2} \sum_{m_x,m_y,m_z} C(m_x, m_y, m_z) \left|F(m_x, m_y, m_z; i)\right|^2 - \frac{\pi}{2\kappa^2 V} \left(\sum_{\alpha} q_{i\alpha}\right)^2$$

The real-space part is common between the full Ewald and PME.

When the system is the reference solvent and the solute is treated as a test particle, $Q(k_x, k_y, k_z; i)$ and $F(m_x, m_y, m_z; i)$ are evaluated only once for each solvent molecule at each solvent configuration for which the test-particle insertion is performed.
For the solute, $Q(k_x, k_y, k_z; i)$ and $F(m_x, m_y, m_z; i)$ are evaluated at each insertion.
$F(m_x, m_y, m_z; i)$ is obtained from $Q(k_x, k_y, k_z; i)$ by 3D-FFT.

The above procedure is problematic in memory. $Q(k_x, k_y, k_z; i)$ and $F(m_x, m_y, m_z; i)$ are $K_x K_y K_z$ in size for each molecule. The memory required for a CPU is $P K_x K_y K_z$, where P is the number of molecules which the CPU is responsible for. Since $K_x K_y K_z$ is 10^5-10^6, the memory requirement is prohibitive if P is more than ~10^2.

Now I define a better procedure for the pair energy in PME. This is the scheme I use in my program.

The mesh size is K_x, K_y, and K_z along the x, y, and z directions, respectively.

The scaled fractional coordinate for a position (x, y, z) is given by

$$(u_x, u_y, u_z) = (xK_x/L_x, yK_y/L_y, zK_z/L_z).$$

The grid point in the space is specified by integers (k_x, k_y, k_z) with

$$0 \leqq k_x \leqq K_x - 1,\ 0 \leqq k_y \leqq K_y - 1,\ 0 \leqq k_z \leqq K_z - 1.$$

The spline function is denoted by $M_n(s)$ and is given by

$$M_n(s) = \frac{1}{(n-1)!}\sum_{i=0}^{n}(-1)^i \frac{n!}{i!(n-i)!}\left[\max(s-i,0)\right]^{n-1}$$
$$= \sum_{i=0}^{[s]}(-1)^i \frac{n}{i!(n-i)!}(s-i)^{n-1}$$

The smoothed charge is expressed as

$$Q(k_x,k_y,k_z;i) = \sum_{\alpha} q_{i\alpha} M_n(u_{i\alpha x} - [u_{i\alpha x}] + l_x) M_n(u_{i\alpha y} - [u_{i\alpha y}] + l_y) M_n(u_{i\alpha z} - [u_{i\alpha z}] + l_z)$$

when k = modulo([u]-l, K) (l = 0, … , n-1) holds for all the x, y, and z components and $Q(k_x, k_y, k_z;i)$ is zero otherwise.

The Fourier transform $F(m_x, m_y, m_z; i)$ of the array $Q(k_x, k_y, k_z; i)$ is expressed as

$$F(m_x,m_y,m_z;i)$$
$$= \sum_{k_x=0}^{K_x-1}\sum_{k_y=0}^{K_y-1}\sum_{k_z=0}^{K_z-1} Q(k_x,k_y,k_z;i)\exp\left[-2\pi i\left(\frac{m_x k_x}{K_x} + \frac{m_y k_y}{K_y} + \frac{m_z k_z}{K_z}\right)\right]$$

(m_x, m_y, m_z) is integer with $0 \leqq m_x \leqq K_x - 1$, $0 \leqq m_y \leqq K_y - 1$, $0 \leqq m_z \leqq K_z - 1$

A function D is defined as

$$D(m_x,m_y,m_z) = \frac{1}{\pi V}\frac{\exp(-\pi^2 \mathbf{m}^2/\kappa^2)}{\mathbf{m}^2}\prod_{j=x,y,z}\frac{1}{\left|\sum_{h=0}^{n-2} M_n(h+1)\exp(-2\pi i h m_j/K_j)\right|^2}$$

where $\mathbf{m} = (m_x'/L_x, m_y'/L_y, m_z'/L_z)$ with

$m' = m$ when $m \leqq K/2$ and $m' = m - K$ when $m > K/2$

and $D = 0$ at $m_x = m_y = m_z = 0$

The reciprocal part of the pair energy between the i-th and j-th particles ($i \neq j$) is

$$\sum_{m_x,m_y,m_z} D(m_x,m_y,m_z)F(m_x,m_y,m_z;i)F^*(m_x,m_y,m_z;j)$$

$$= \sum_{m_x,m_y,m_z} D(m_x,m_y,m_z)F^*(m_x,m_y,m_z;j)\left\{\sum_{k_x,k_y,k_z} Q(k_x,k_y,k_z;i)\exp\left[-2\pi i\left(\frac{m_x k_x}{K_x}+\frac{m_y k_y}{K_y}+\frac{m_z k_z}{K_z}\right)\right]\right\}$$

$$= \sum_{k_x,k_y,k_z} Q(k_x,k_y,k_z;i)\sum_{m_x,m_y,m_z} D(m_x,m_y,m_z)F^*(m_x,m_y,m_z;j)\exp\left[-2\pi i\left(\frac{m_x k_x}{K_x}+\frac{m_y k_y}{K_y}+\frac{m_z k_z}{K_z}\right)\right]$$

$$= \sum_{\alpha} q_{i\alpha}\sum_{l_x=0}^{n-1}\sum_{l_y=0}^{n-1}\sum_{l_z=0}^{n-1}\left[\begin{array}{l} M_n(u_{i\alpha x}-[u_{i\alpha x}]+l_x)M_n(u_{i\alpha y}-[u_{i\alpha y}]+l_y)M_n(u_{i\alpha z}-[u_{i\alpha z}]+l_z) \\ \sum_{m_x,m_y,m_z} D(m_x,m_y,m_z)F^*(m_x,m_y,m_z;j)\exp\left[-2\pi i\left(\frac{m_x k_x}{K_x}+\frac{m_y k_y}{K_y}+\frac{m_z k_z}{K_z}\right)\right]\end{array}\right]$$

where k is set to k = modulo([u]-l, K) in the last line, and the exponential term is

$$\exp\left[-2\pi i\left(\frac{m_x([u_{i\alpha x}]-l_x)}{K_x}+\frac{m_y([u_{i\alpha y}]-l_y)}{K_y}+\frac{m_z([u_{i\alpha z}]-l_z)}{K_z}\right)\right]$$

When the Fourier-transform of $D(m_x, m_y, m_z)F(m_x, m_y, m_z; j)^*$ is denoted by $E(m_x, m_y, m_z; j)$ as

$$E(k_x,k_y,k_z;j) = \sum_{m_x,m_y,m_z} D(m_x,m_y,m_z)F^*(m_x,m_y,m_z;j)\exp\left[-2\pi i\left(\frac{m_x k_x}{K_x}+\frac{m_y k_y}{K_y}+\frac{m_z k_z}{K_z}\right)\right]$$

the Coulombic part of the pair energy between the i-th and j-th particles ($i \neq j$) is

$$\sum_{\alpha,\gamma} q_{i\alpha}q_{j\gamma}\frac{\mathrm{erfc}(\kappa r_{i\alpha.j\gamma})}{r_{i\alpha.j\gamma}}$$
$$+\sum_{\alpha} q_{i\alpha}\sum_{l_x=0}^{n-1}\sum_{l_y=0}^{n-1}\sum_{l_z=0}^{n-1}\left[\begin{array}{l} M_n(u_{i\alpha x}-[u_{i\alpha x}]+l_x)M_n(u_{i\alpha y}-[u_{i\alpha y}]+l_y)M_n(u_{i\alpha z}-[u_{i\alpha z}]+l_z) \\ E([u_{i\alpha x}]-l_x,[u_{i\alpha y}]-l_y,[u_{i\alpha z}]-l_z)\end{array}\right]-\frac{\pi}{\kappa^2 V}\left(\sum_{\alpha} q_{i\alpha}\right)\left(\sum_{\beta} q_{j\beta}\right)$$

The self-energy of the i-th particle is

$$\sum_{(\alpha,\gamma)\notin M} q_{i\alpha}q_{i\gamma}\frac{\mathrm{erfc}(\kappa r_{i\alpha.i\gamma})}{r_{i\alpha.i\gamma}}-\sum_{(\alpha,\gamma)\in M} q_{i\alpha}q_{j\gamma}\frac{\mathrm{erf}(\kappa r_{i\alpha.i\gamma})}{r_{i\alpha.i\gamma}}-\frac{\kappa}{\sqrt{\pi}}\sum_{\alpha} q_{i\alpha}{}^2$$
$$+\frac{1}{2}\sum_{m_x,m_y,m_z} C(m_x,m_y,m_z)\left|F(m_x,m_y,m_z;i)\right|^2-\frac{\pi}{2\kappa^2 V}\left(\sum_{\alpha} q_{i\alpha}\right)^2$$

The real-space part is common between the full Ewald and PME.

With the above procedure, the calculation time for the reciprocal term is negligible compared to the calculation for the real-space term. In other words, with the above procedure, the speedup should now be targeted at the real-space calculations.

The above scheme is the one which is employed in my program, recpcal.f. treating the reciprocal part in the Ewald and PME methods.

4) Functional for the solvation free energy

In this section, I will present the discretized version of the functional for the solvation free energy. First, I summarize the continuous version presented in part II paper, and later I show the discretization.

The system of our interest is a dilute solution containing a single solute molecule. The intermolecular interaction is supposed to be pairwise additive. The complete set of the position and orientation of a solvent (environment) molecule is called the full coordinate and is denoted collectively by **x**. When the solute molecule involves structural flexibility, its intramolecular degrees of freedom are collectively written as ψ. The overall position and orientation of the solute molecule are also incorporated in ψ. The solute-solvent interaction potential of interest is v. It may be expressed as $v^f(\psi,\mathbf{x})$ in the full coordinate representation, where a superscript f is attached to emphasize that a function is represented over the full coordinate.

The solvation free energy $\Delta\mu$ is the free energy change corresponding to the gradual insertion process of the solute molecule. In $\Delta\mu$, only the contribution from the potential energy is involved and the ideal (kinetic) contribution is excluded at the outset. When the intramolecular energy of the solute is $\Psi(\psi)$ and the total solvent-solvent interaction energy is $U(\mathbf{X})$, $\Delta\mu$ is expressed as

$$\exp(-\beta\Delta\mu) = \frac{\int d\psi d\mathbf{X} \exp\left(-\beta\left\{\Psi(\psi) + \sum_i v^f(\psi,\mathbf{x}_i) + U(\mathbf{X})\right\}\right)}{\int d\psi d\mathbf{X} \exp\left(-\beta\left\{\Psi(\psi) + U(\mathbf{X})\right\}\right)} \tag{1}$$

where $\mathbf{X}$ represents the solvent configuration collectively and $\mathbf{x}_i$ is the full coordinate of the ith solvent (environment) molecule. β is the inverse of k_BT with the Boltzmann constant k_B and the temperature T. A restriction of attention to a certain set of solute intramolecular state can be made simply by the corresponding alteration of the domain of integration over ψ. The energy representation is introduced by adopting the value of the solute-solvent interaction v of interest as the coordinate ε for the distribution of the solvent molecule around the solute molecule. The instantaneous distribution $\hat{\rho}^e$ is then defined as

$$\hat{\rho}^e(\varepsilon) = \sum_i \delta\left(v^f(\psi,\mathbf{x}_i) - \varepsilon\right) \tag{2}$$

where the sum is taken over the solvent molecules and a superscript e is attached to

emphasize that a function is represented over the energy coordinate. Note that the specification of v is necessary in Eq. (2).

In the energy representation, the solvation free energy $\Delta\mu$ is approximately expressed in terms of distribution functions constructed from $\hat{\rho}^e$ in the solution and reference solvent systems. In our treatments, the solution system refers to the system in which the solute molecule interacts with the solvent under the solute-solvent interaction v of interest at full coupling. In the solution, the average distribution ρ^e of the v value is relevant in the approximate construction of $\Delta\mu$and is given by

$$\rho^e(\varepsilon) = \left\langle \hat{\rho}^e(\varepsilon) \right\rangle \tag{3}$$

where $\langle \cdots \rangle$ represents the ensemble average in the solution system of interest. On the other hand, the reference solvent system denotes the system in which no interaction is physically present between the solute and solvent (environment) molecules. At an instantaneous configuration of the reference solvent system, $\hat{\rho}^e$ is constructed by placing the solute molecule in the neat solvent system as a test particle. The average distribution ρ_0^e and the correlation matrix χ_0^e then appear in the approximate functional for $\Delta\mu$ and are expressed, respectively, as

$$\rho_0^e(\varepsilon) = \left\langle \hat{\rho}^e(\varepsilon) \right\rangle_0 \tag{4}$$

and

$$\chi_0^e(\varepsilon,\eta) = \left\langle \hat{\rho}^e(\varepsilon)\hat{\rho}^e(\eta) \right\rangle_0 - \left\langle \hat{\rho}^e(\varepsilon) \right\rangle_0 \left\langle \hat{\rho}^e(\eta) \right\rangle_0 \tag{5}$$

where $\langle \cdots \rangle_0$ represents the ensemble average in the reference solvent system. In this case, the solute molecule is placed as a test particle in the neat solvent system, typically using the particle insertion scheme.

An approximate functional for $\Delta\mu$ is derived in part II paper. $\Delta\mu$ is given by a set of definitions and equations listed as

$$w^e(\varepsilon) = -k_B T \log\left(\frac{\rho^e(\varepsilon)}{\rho_0^e(\varepsilon)}\right) - \varepsilon - C \tag{6}$$

$$w_0^e(\varepsilon) = -k_B T \int d\eta \left(\frac{\delta(\varepsilon-\eta)}{\rho_0^e(\varepsilon)} - \left(\chi_0^e\right)^{-1}(\varepsilon,\eta)\right)\left(\rho^e(\eta) - \rho_0^e(\eta)\right) - C_0 \tag{7}$$

$$C = \frac{1}{\int d\varepsilon \rho^e(\varepsilon)} \int d\varepsilon \rho^e(\varepsilon)\left[-k_B T \log\left(\frac{\rho^e(\varepsilon)}{\rho_0^e(\varepsilon)}\right) - \varepsilon\right] \tag{8}$$

$$\begin{aligned} C_0 &= \frac{1}{\int d\varepsilon \rho_0^e(\varepsilon)} \int d\varepsilon \rho_0^e(\varepsilon)\left[-k_B T \int d\eta \left(\frac{\delta(\varepsilon-\eta)}{\rho_0^e(\varepsilon)} - \left(\chi_0^e\right)^{-1}(\varepsilon,\eta)\right)\left(\rho^e(\eta) - \rho_0^e(\eta)\right)\right] \\ &= \frac{k_B T}{\int d\varepsilon \rho_0^e(\varepsilon)} \int d\varepsilon d\eta \rho_0^e(\varepsilon)\left(\chi_0^e\right)^{-1}(\varepsilon,\eta)\left(\rho^e(\eta) - \rho_0^e(\eta)\right) \end{aligned} \tag{9}$$

$$\begin{aligned} \Delta\mu = \int d\varepsilon \rho(\varepsilon) - k_B T \int d\varepsilon \Big[&\left(\rho^e(\varepsilon) - \rho_0^e(\varepsilon)\right) - \rho^e(\varepsilon) \log\left(\frac{\rho^e(\varepsilon)}{\rho_0^e(\varepsilon)}\right) \\ &- \{\alpha(\varepsilon) F(\varepsilon) + (1-\alpha(\varepsilon)) F_0(\varepsilon)\}\left(\rho^e(\varepsilon) - \rho_0^e(\varepsilon)\right)\Big] \end{aligned} \tag{10}$$

$$F(\varepsilon) = \begin{cases} \beta w^e(\varepsilon) + 1 + \dfrac{\beta w^e(\varepsilon)}{\exp\left(-\beta w^e(\varepsilon)\right) - 1} & (\text{when } w^e(\varepsilon) \le 0) \\ \dfrac{1}{2}\beta w^e(\varepsilon) & (\text{when } w^e(\varepsilon) \ge 0) \end{cases} \tag{11}$$

$$F_0(\varepsilon) = \begin{cases} -\log\left(1 - \beta w_0^e(\varepsilon)\right) + 1 + \dfrac{\log\left(1 - \beta w_0^e(\varepsilon)\right)}{\beta w_0^e(\varepsilon)} & (\text{when } w_0^e(\varepsilon) \le 0) \\ \dfrac{1}{2}\beta w_0^e(\varepsilon) & (\text{when } w_0^e(\varepsilon) \ge 0) \end{cases} \tag{12}$$

$$\alpha(\varepsilon) = \begin{cases} 1 & (\text{when } \rho^e(\varepsilon) \ge \rho_0^e(\varepsilon)) \\ 1 - \left(\dfrac{\rho^e(\varepsilon) - \rho_0^e(\varepsilon)}{\rho^e(\varepsilon) + \rho_0^e(\varepsilon)}\right)^2 & (\text{when } \rho^e(\varepsilon) \le \rho_0^e(\varepsilon)) \end{cases} \tag{13}$$

Note that the first term of Eq. (10) is the average sum of the solute-solvent interaction energy. The rest of Eq. (10) is constructed from the distribution functions.

C and C_0 of Eqs. (8) and (9) were simply set to zero in my previous works. Actually, they are of order $1/N$, where N is the number of particles in the system, and it is safely

set to zero as far as N is ~10^2. The form of Eqs. (8) and (9) is adopted so that the functional is exact in the limit of N = 1. In other words, with C and C_0 of Eqs. (8) and (9), the results in the previous works are virtually unchanged and the forthcoming results done at small N will be better.

Now I turn to the discretized version. First thing to do is to introduce the discretized coordinate for the solute-solvent pair energy. The way I did is described in Appendix of paper II. There, the pair energy is discretized linearly up to some energy value $\varepsilon_{\mathrm{core}}$ and beyond it, the discretization is done logarithmically. $\varepsilon_{\mathrm{core}}$ is chosen so that it is larger than the largest solute-solvent pair energy in the solution system (NOT the reference solvent with particle insertion) with nonzero $\rho^e(\varepsilon)$. Typically, I have a bunch of arrays of solute-solvent pair energies in the solution. Then I pick up the largest value among them and let $\varepsilon_{\mathrm{core}}$ be larger than that the picked, largest one.

The discretized version $\hat{n}^e(i)$ of instantaneous distribution for the ith region of the energy coordinate is simply the instantaneous number of solute-solvent pair interaction energies contained in the ith region. When I am given an instantaneous array of solute-solvent pair interaction energies, I simply count the number of pair energies which are contained in the ith region of the discretized energy coordinate, and the number is equal to $\hat{n}^e(i)$. $\hat{n}^e(i)$ is thus the histogram constructed from an array of solute-solvent pair interaction energies. For each instantaneous array, $\hat{n}^e(i)$ is integer. Eqs (3)-(5) are then simply modified to

$$n^e(i) = \left\langle \hat{n}^e(i) \right\rangle \tag{14}$$

$$n_0^e(i) = \left\langle \hat{n}_0^e(i) \right\rangle_0 \tag{15}$$

and

$$\chi_0^e(i,j) = \left\langle \hat{n}^e(i)\hat{n}^e(j) \right\rangle_0 - \left\langle \hat{n}^e(i) \right\rangle_0 \left\langle \hat{n}^e(j) \right\rangle_0 \tag{16}$$

where $\langle \cdots \rangle$ represents the ensemble average in the solution system of interest and

$\langle \cdots \rangle_0$ represents the ensemble average in the reference solvent system. I will use $n^e(i)$, $n_0^e(i)$, and $\chi_0^e(i,j)$, as defined in Eqs (14)-(16). I do not have to normalized them (divide them by the width of the dicretized energy interval), except when I want to show the distribution functions for some presentation purposes.

The expression for $\Delta\mu$ is modified with introduction of discretized energy coordinate and given by

$$w^e(i) = -k_B T \log\left(\frac{n^e(i)}{n_0^e(i)}\right) - \varepsilon_i - C \tag{17}$$

$$C = \frac{1}{\sum_i n^e(i)} \sum_i n^e(i) \left[-k_B T \log\left(\frac{n^e(\varepsilon)}{n_0^e(\varepsilon)}\right) - \varepsilon_i \right] \tag{18}$$

(ε_i is the midpoint value of the ith region of the energy coordinate)

$$w_0^e(i) = -k_B T \sum_j \left(\frac{\delta_{ij}}{n_0^e(i)} - \left(\chi_0^e\right)^{-1}(i,j) \right) \left(n^e(j) - n_0^e(j) \right) - C_0 \tag{19}$$

(δ_{ij} is the Kronecker delta; = 1 when i=j, and =0 when i and j are not the same)

$$\begin{aligned} C_0 &= \frac{1}{\sum_i n_0^e(i)} \sum_i n_0^e(i) \left[-k_B T \sum_j \left(\frac{\delta_{ij}}{n_0^e(i)} - \left(\chi_0^e\right)^{-1}(i,j) \right) \left(n^e(j) - n_0^e(j) \right) \right] \\ &= \frac{k_B T}{\sum_i n_0^e(i)} \sum_i n_0^e(i) \left(\chi_0^e\right)^{-1}(i,j) \left(n^e(j) - n_0^e(j) \right) \end{aligned} \tag{20}$$

$$\begin{aligned} \Delta\mu = &\left(\text{average sum of the solute-solvent interaction energy}\right) \\ &- k_B T \sum_i \left[\begin{array}{l} \left(n^e(i) - n_0^e(i) \right) - n^e(i) \log\left(\frac{n^e(i)}{n_0^e(i)}\right) \\ - \left\{ \alpha(i) F(i) + (1-\alpha(i)) F_0(i) \right\} \left(n^e(i) - n_0^e(i) \right) \end{array} \right] \end{aligned} \tag{21}$$

$$F(i)=\begin{cases}\beta w^e(i)+1+\dfrac{\beta w^e(i)}{\exp\left(-\beta w^e(i)\right)-1} & (\text{when } w^e(i)\le 0)\\ \dfrac{1}{2}\beta w^e(i) & (\text{when } w^e(i)\ge 0)\end{cases} \tag{22}$$

$$F_0(i)=\begin{cases}-\log\left(1-\beta w_0^e(i)\right)+1+\dfrac{\log\left(1-\beta w_0^e(i)\right)}{\beta w_0^e(i)} & (\text{when } w_0^e(i)\le 0)\\ \dfrac{1}{2}\beta w_0^e(i) & (\text{when } w_0^e(i)\ge 0)\end{cases} \tag{23}$$

$$\alpha(i)=\begin{cases}1 & (\text{when } n^e(i)\ge n_0^e(i))\\ 1-\left(\dfrac{n^e(i)-n_0^e(i)}{n^e(i)+n_0^e(i)}\right)^2 & (\text{when } n^e(i)\le n_0^e(i))\end{cases} \tag{24}$$

In. Eq. (21), the first term can be expressed as

$$\left(\text{average sum of the solute-solvent interaction energy}\right)=\sum_i \varepsilon_i n^e(i) \tag{25}$$

In practical implementation, however, I do not use Eq. (25). I simply take (average sum of the solute-solvent interaction energy) from the simulation, and do not construct it from the distribution function $n^e(i)$. When the energy coordinate is discretized, the discretization error arises. When Eq. (25) is used with the discretized distribution function $n^e(i)$, it also carries the discretization error. An experience is actually that most of the discretization error comes from Eq. (25) when it is used with Eqs. (14)-(24). Thus, when (average sum of the solute-solvent interaction energy) is taken from the simulation, much of the discretization error vanishes. This is the reason why I do not use Eq. (25) and take (average sum of the solute-solvent interaction energy) directly from the simulation.

In Eqs. (19)-(20), the inversion of χ_0^e needs to be done. The procedure is described in Appendix B of part III paper, but I want to summarize it here. χ_0^e is a symmetric matrix. I first determine the eigenvalue and eigenvector. In my program, I use a LAPACK driver. Let z_l be the lth eigenvalue and $g_l(i)$ be the associated eigenvector. Then

$$\sum_j \chi_0^e(i,j) g_l(j) = z_l g_l(i)$$

As described in Appendix B of part III paper, z_l is positive except for the smallest one. The smallest one is called z_0 and Eq. (16) is rewritten as

$$w_0^e(i) = -k_B T \frac{\left(n^e(i) - n_0^e(i)\right)}{n_0^e(i)} - \bar{u}_0^e(i)$$

$$\bar{u}_0^e(i) = C - k_B T \sum_{l \neq 0} g_l(i) \frac{1}{z_l} \sum_k g_l(k) \left(n^e(k) - n_0^e(k)\right) \tag{26}$$

and the constant C is canceled by Eq. (20).

The numerical point is the normalization of $n^e(i)$ $n_0^e(i)$ and $\chi_0^e(i,j)$. The procedure is described in Appendix C of part III paper. The normalization is simple for $n^e(i)$ and $n_0^e(i)$. Let N be the number of total solvent (environment) molecules and $A = \sum_i n^e(i)$ and $A_0 = \sum_i n_0^e(i)$ for the computationally obtained $n^e(i)$ and $n_0^e(i)$, then $\frac{N}{A} n^e(i)$ and $\frac{N}{A_0} n_0^e(i)$ be the new $n^e(i)$ and $n_0^e(i)$, respectively, which satisfy Eq (C1) of the part II paper and are used to calculate $\Delta\mu$. For $\chi_0^e(i,j)$, an iterative procedure is necessary. According to Eq. (16), $\chi_0^e(i,j)$ is written as

$$n^{(2)}(i,j) = \left\langle \hat{n}^e(i) \hat{n}^e(j) \right\rangle_0$$

$$\chi_0^e(i,j) = n^{(2)}(i,j) - \left\langle \hat{n}^e(i) \right\rangle_0 \left\langle \hat{n}^e(j) \right\rangle_0 \tag{27}$$

Here, $n^{(2)}(i,j)$ is defined by the first line of Eq. (27). By definition, it needs to satisfy

$$\sum_j n^{(2)}(i,j) = N\, n_0^e(i) \tag{28}$$

where N is the number of total solvent (environment) molecules. But due to the finite

precision of the numerical representation of $n^{(2)}(i,j)$, Eq. (28) is not necessarily satisfied. We then scale $n^{(2)}(i,j)$ to enforce the condition Eq. (28). We determine $R(i)$ which satisfies

$$\sum_j R(i)R(j)n^{(2)}(i,j) = N\, n_0^e(i) \tag{29}$$

To solve this, set $R(i) = 1 + S(i)$, where $S(i)$ is supposed to be quite small. Then rewrite Eq. (29) as

$$S(i) = \frac{N n_0^e(i)}{\sum_j (1+S(j)) n^{(2)}(i,j)} - 1 \tag{30}$$

The iteration starts by setting $S(i) = 0$ first. Then calculate the right-hand side of Eq. (30) and substitute for the left-hand side. I do this until convergence. When the double precision is used for express $n^e(i)$ $n_0^e(i)$ and $\chi_0^e(i,j)$, the convergence is achieved very fast or even the iteration is not necessary. When $n_0^e(i) = 0$ actually, the value of $S(i)$ is not necessary to be determined. Indeed, when $n_0^e(i) = 0$, the sum over j is also zero and $S(i)$ is undetermined. But there is no contribution to $\Delta\mu$ from such i. Once $R(i)$ is determined from the above procedure, $R(i)R(j)n^{(2)}(i,j)$ now serves as the new $n^{(2)}(i,j)$ and is used to construct $\chi_0^e(i,j)$ by the second line of Eq. (27).

When $n^e(i) = 0$, I do not have to worry about the log in Eq. (17). $\alpha(i) = 0$ for such i, anyway, so that I do not care the value of $w^e(i)$. When $n^e(i) = n_0^e(i) = 0$, such i does not make any contribution and I do not have to care about it, either, The only problematic case is $n^e(i) \neq 0$ and $n_0^e(i) = 0$. In this case, it is not possible to determine

the value of $w^e(i)$, while the value is needed in Eqs. (21) and (22). I can simply skip such *i*, as described in the Appendix of the part II paper, simply because the probability of occurrence is quite low. Or I can use the extra- or interpolation procedure described in footnote 40 of the part III paper. The point is simply to pick up the *j* closest to *i* for which $n^e(j) \neq 0$ and $n_0^e(j) \neq 0$ and $w^e(j)$ can be determined and to set $w^e(i) = w^e(j)$.

5) Treatment of the self-energy in Ewald and PME methods

As described above, the solute interacts with its own images in the Ewald and PME methods. We call this part the self-energy $\varepsilon^{\mathrm{self}}$. And we call the pair interaction energy between the solute and the *i*th solvent molecule $\varepsilon^{\mathrm{pair}}{}_i$. Actually, the energy difference between the solution and reference solvent systems reads

$$\varepsilon^{\mathrm{self}} + \sum_i \varepsilon^{\mathrm{pair}}{}_i$$

at each configuration. In this equation, $\varepsilon^{\mathrm{self}}$ is a function of the solute configuration and $\varepsilon^{\mathrm{pair}}{}_i$ is a function of the solute configuration and the configuration of the *i*th solvent molecule. In the *NPT* ensemble, they also depend on the system volume. The solvation free energy $\Delta\mu$ is given by

$$\exp(-\beta\Delta\mu) = \left\langle \exp\left(-\beta\left\{\varepsilon^{\mathrm{self}} + \sum_i \varepsilon^{\mathrm{pair}}{}_i\right\}\right)\right\rangle_0$$

where the average $<\ldots>_0$ is taken in the reference solvent system. When we evaluate $\Delta\mu$, we actually rewrite the equation as

$$\Delta\mu = -k_B T \log\left\langle \exp\left(-\beta\varepsilon^{\mathrm{self}}\right)\right\rangle_0 - k_B T \log\left(\frac{\left\langle \exp\left(-\beta\left\{\varepsilon^{\mathrm{self}} + \sum_i \varepsilon^{\mathrm{pair}}{}_i\right\}\right)\right\rangle_0}{\left\langle \exp\left(-\beta\varepsilon^{\mathrm{self}}\right)\right\rangle_0}\right)$$

The first term is an easy calculation in the reference solvent system. We can simply calculate the distribution function of $\varepsilon^{\mathrm{self}}$, and use the equation above. What I calculate in my program is the second term. In this case, $<\ldots>_0$ is actually replaced by

$$\frac{\left\langle \ldots \exp\left(-\beta\varepsilon^{\mathrm{self}}\right)\right\rangle_0}{\left\langle \exp\left(-\beta\varepsilon^{\mathrm{self}}\right)\right\rangle_0}$$

What I mean by $<\ldots>_0$ is actually this, modified average. If the variation of $\varepsilon^{\mathrm{self}}$ against the change in the configuration is much smaller than $k_B T$, the modification does not have any effect. Typically, when the solute is neutral and its size is much smaller than the system size, the modification of the averaging procedure changes the $\Delta\mu$ value only within the error bar. The first term is also negligible when the solute is neutral and its size is much smaller than the system size, while it is significant in magnitude when the solute is ionic.

6) Standard mesh size

The energy coordinate ε needs to be discretized in practice. The standard mesh size currently used is

in kcal/mol
0.05 between -30.0 and -0.2
0.002 between -0.2 and -0.02
0.0002 between -0.02 and 0.02
0.002 between 0.02 and 0.2
0.05 between 0.2 and 20.0
logarithmic mesh between 20.0 and 1.0e12
  number of logarithmic meshes is 300
  when the calculated ε is larger than 1.0e12,
  it is given to the mesh with the largest energy

The smallest energy -30.0 depends on the strength of the solute-solvent interaction.
It needs to be smaller than the minimum value of the solute-solvent interaction.

With the above "default", the mesh-size dependence is typically negligible (less than 0.1 kcal/mol). It is examined with the procedure given around Equation (A5) in Appendix of my part II paper; I usually examine K of Eq. (A5) up to K = 10.

When the mesh-size dependence is significant, the first thing to do is to increase the number of insertions.
The mesh-size error comes from ρ_0 and χ_0.
The statistical error (error due to short simulation) comes from ρ.
I still do not understand why this is the general case, but this is a rule I often observe.

7) The scheme to read a trajectory, obtain the energy distribution functions using ermod program, and calculate the solvation free energy

The general procedure is

1) use an MD program to generate a trajectory and prepare several files
2) compile the ermod program and run it to obtain the distribution functions
3) calculate the solvation free energy from the distribution functions

Each step will be described below.

The followings are described for the trajectory-reading case. When the ermod program is combined on the source-file level with the parent MD program, the manual setting procedure is minimal (probably). For the source-file combination, contact me.

a) solution system

trajectory file: rename (or soft-link) it to HISTORY

in the case of NAMD: DCD format is available

in the case of GROMACS: g96 format is available

see the setconf.f program for the g96 format.

in the case of MARBLE, DL_POLY, and CHARMM: text format is available

see the setconf.f program for the format.

system parameter file: MDinfo file necessary except in the case of CHARMM

example case of ethylbenzene in water

```
> cat MDinfo
10000    2
2000     1
3        18
```

1st line: number of snapshot configurations and number of molecular species

2nd line: number of molecules of each species

3rd line: number of sites within each solvent species

in the case of NAMD:

rename (or soft-link) the cell parameter file (xst file) to HISTCELL

each snapshot contained in HISTORY and HISTCELL files needs to be correspondent

interaction parameter file

for solute: SltInfo

list the atomic species and the interaction parameters

example case

```
1   C   -0.1150     0.7000E-01 3.5500532
2   H    0.1150     0.3000E-01 2.4200373
3   C   -0.1150     0.7000E-01 3.5500532
…..
```

1st column: numbering the atomic site

2nd column: element symbol for the atomic site

3rd column: partial charge on the site in the unit of elementary charge

4th column: LJ energy parameter ε

5th column: LJ length parameter σ or Rmin/2

The units and format of the LJ energy and length parameters are specified by the parameter ljformat. See below for the explanation for ljformat. The default is that the energy parameter is the LJ ε in the unit of kcal/mol and that the length parameter is the Rmin/2 in the unit of Å. The above example is expressed with the LJ ε in the unit of kcal/mol and the LJ σ in the unit of Å, corresponding to the standard expression $4\varepsilon[(\sigma/r)^{12}-(\sigma/r)^{6}]$. In this case, it is necessary to set ljformat = 0.

for solvent species: MolPrm1, MolPrm2 …

For the 1st solvent species MolPrm1

For the 2nd solvent species MolPrm2 …

Example for (CHARMM-modified) TIP3P water

```
1 O    -0.8340    0.1521    3.1506
2 H     0.4170    0.0460    0.4000
3 H     0.4170    0.0460    0.4000
```

The format is the same as SltInfo described above

b) reference solvent system

trajectory file: HISTORY

system parameter file: MDinfo file necessary except in the case of CHARMM

cell parameter file in the case of NAMD: HISTCELL

The format and details are the same as those described for the solution system

interaction parameter file

for solvent species: MolPrm1, MolPrm2 …

same as the one for the solution system

for solute:

b1) when the solute is flexible

file: SltInfo same as the one for the solution system

There also needs one more trajectory for the single-molecule simulation for the solute (see below)

b2) when the solute is rigid

file: SltInfo the format is different from the one for the solution

example case

1	C	-0.115	7.00E-02	3.5500532	1.457	0.477	-18.403
2	H	0.115	3.00E-02	2.4200373	2.345	0.202	-17.853
3	C	-0.115	7.00E-02	3.5500532	0.268	-0.215	-18.094

…..

The format is similar to the cases in the solution system and b1) above

1st column: numbering the atomic site

2nd column: element symbol for the atomic site

3th column: partial charge on the site in the unit of elementary charge

4th column: LJ energy parameter ε

5th column: LJ length parameter σ or Rmin/2

See the description in a) solution system for the unit and format

6rd to 8th columns: intramolecular (x, y, z) coordinate in the unit of Å

The center-of-mass position or orientation does not need to be "standardized". For example, the center of mass needs not to be set to (0,0,0)

c) single-molecule trajectory for the solute

needed only when the solute is flexible

Run MD or MC of the solute only (no solvent) and save the trajectory

The system parameters (e.g. temperature) to obtain the trajectory need to be the same as those for the solution and reference solvent systems.

trajectory file: rename (or soft-link) it to SltConf

A) To obtain the energy distribution function $\rho^e(\varepsilon)$ in solution

$ mkdir soln

$ cd soln

Move the HISORY, MDinfo, HISTCELL, MolPrm1, MolPrm2, …, and SltInfo (for those which exist) to soln directory

Run the executable constructed from ermod to obtain the distribution function

To construct the executable, see below

B) To obtain the energy distribution functions $\rho^e{}_0(\varepsilon)$ and $\chi^e{}_0(\varepsilon,\eta)$ in reference solvent

$ mkdir refs

$ cd refs

Move the HISORY, MDinfo, HISTCELL, MolPrm1, MolPrm2, …, SltInfo, and SltConf (for those which exist) to refs directory

Run the executable constructed from ermod to obtain the distribution functions

To construct the executable, see below

The parameter names and meanings of the ermod program are (mostly) written in the head of enganal.f program.

The parameters are set in the param_eng file.

The ermod program is compiled every time used.

The typical parameter settings are described below.

param_eng file

iseed: random number seed arbitrary, and 9 digits or less

slttype: 1 for the solution system
2 for the reference solvent system with rigid solute
3 for the reference solvent system with flexible solute

corrcal : calculation of the correlation matrix

0 : no calculation 1 : calculation performed

default = 0 when slttype = 1

default = 1 when slttype = 2 or 3

skpcnf: interval to skip the configuration examined

unless explicitly stated in param_eng, skpcnf = 1 in the trajectory reading

sltpick: specifying the solute species

if the system contains numtype molecular species, 1 <= sltpick <= numtype

When slttype = 1, sltpick is specified according to 1 <= sltpick <= numtype.
If not specified, sltpick = 1.

When slttype = 2 or 3, it is not necessary to specify sltpick.

It is automatically equal to numtype

estype: type of system 1 : constant volume 2 : constant pressure

boxshp: shape of the unit cell box 0 : non-periodic 1 : periodic and rectangular

inscnd: geometrical condition of the solute configuration

0 : random 1 : spherical 2 : slab 3 : reference

inscfg: position and orientation for the inserted solute

0 : only the intramolecular configuration is from the file (default)

1 : orientation is fixed from the file with random position

2 : position and orientation are also fixed from the file

The file for the solute configuration

is SltInfo when slttype = 2 and is SltConf when slttype = 3

hostspec: molecular type forming the host (micelle, membrane, or protein)

active only when inscnd is not equal to 0

default value = 1 and set to 1 if not specified

1 <= hostspec <= numtype when slttype = 1

1 <= hostspec <= numtype-1 when slttype = or > 2

ljformat : input-file format for the LJ energy and length parameters

0 : ε (kcal/mol) and σ (Å) (standard LJ)

1 : ε (kcal/mol) and Rmin/2 (Å) (CHARMM and AMBER expressions)

2 : ε (kJ/mol) and σ (nm) (GROMACS expression)

3 : C6 (kcal/mol Å ^{6}) and C12 (kcal/mol Å ^{12}) (OPLS expression)

4 : C6 (kJ/mol nm^{6}) and C12 (kJ/mol nm^{12})

default value = 1

inptemp: temperature of the system in Kelvin

intprm: whether the intereaction paramters given in subroutine OUTintprm of setconf.f are read from the parent MD program

default = 0 in the case of on-the-fly calculation

default = 1 in the case of trajectory reading

estype, boxshp, and inptemp need to be set to the values in the parent MD

When the parent MD is performed in the NVE ensemble (in an ensemble without temperature control), inptemp is the average temperature.

lwreg, upreg: active only when inscnd = 1, 2, or 3 (no meaning when inscnd = 0)

If inscnd = 1, the solute is located in the region of lwreg < r < upreg, where r is

the distance from the center of the spherical object, typically micelle.

If inscnd = 2, the solute is located in the region of lwreg < z < upreg, where z is the distance along the normal direction from the center of the planar object, typically membrane

If inscnd = 3, the solute is located in the condition of lwreg < RMSD < upreg, where RMSD is the root-mean-square deviation from the reference structure, typically used for protein system

The unit is Å

engdiv: number of divisions (blocks) of the total simulation length

if engdiv = 1

output $\rho^e(\varepsilon)$ for the solution run is the file engsln.tt

output $\rho^e{}_0(\varepsilon)$ and $n^{(2)}(\varepsilon,\eta)$ for the reference solvent run are the files engref.tt and corref.tt

if engdiv > 1 and engdiv = n

output $\rho^e(\varepsilon)$ for the solution run is the files engsln.01, engsln.02, …engsln.n

output $\rho^e{}_0(\varepsilon)$ and $n^{(2)}(\varepsilon,\eta)$ for the reference solvent run is the files engref.01, engref.02, …engref.n, corref.01, corref.02, …corref.n

engdiv needs not be the same for the solution and reference solvent runs

Note that the output is $n^{(2)}(\varepsilon,\eta)$, instead of $\chi^e{}_0(\varepsilon,\eta)$ in the ermod program

$\chi^e{}_0(\varepsilon,\eta)$ is obtained as $\chi^e{}_0(\varepsilon,\eta)=n^{(2)}(\varepsilon,\eta)-\rho^e{}_0(\varepsilon)\rho^e{}_0(\eta)$ in the program to calculate the solvation free energy.

For the solution run, the average sum of the solute-solvent interaction energy is given as an output in the file aveuv.tt. In this file, the average sum for each solvent species is given for each block (division) of the simulation

maxins: number of insertions per configuration for test solute particle in the reference solvent system

if not specified, maxins=1000

cmbrule : combination rule for LJ interaction

= 0 when arithmetic mean is used for LJ sigma as for AMBER and CHARMM

= 1 when geometric mean is used for LJ sigma as for OPLS

geometric mean is always used for LJ epsilon

cltype: treatment of Coulomb interaction 0 : bare 1 : Ewald 2 : PME
screen: screening constant in Ewald or PME
ewtoler: Ewald and PME tolerance to calculate the screen parameter
when screen is given, screen has the priority and ewtoler is neglected
elecut: cutoff of the real-space part of electrostatic interaction
lwljcut: lower limit of the LJ cutoff tapering function
upljcut: upper limit of the LJ cutoff tapering function
splodr: order of spline function used in PME
ew1max,ew2max,ew3max: number of reciprocal vectors along one direction
if not specied, ew2max = ew1max and ew3max = ew1max
ms1max,ms2max,ms3max: number of meshes in PME along one direction
if not specied, ms2max = ms1max and ms3max = ms1max

cltype, screen, elecut, lwljcut, upljcut, splodr, ew1max, ew2max, ew3max, ms1max, ms2max, and ms3max need to be set to the values in the parent MD

peread : determines whether the parameters are read from a separate file
--- 0 : parameters are read from param_eng (default)
--- 1 : parameters are read separately from a file EcdInfo
The format of the EcdInfo file can be seen in enginit subroutine
ecdmin: minimum value of the solute-solvent energy
ecfmns: smaller side of the finely dicretized solute-solvent energy
ecmns0: smaller side of the very finely dicretized energy near ecdcen
ecdcen: central value of the energy coordinate, typically zero
ecpls0: larger side of the very finely dicretized energy near ecdcen
ecfpls: larger side of the finely dicretized solute-solvent energy
eccore: the solute-solvent energy at which the dicretization is changed
ecdmax: maximum value of the solute-solvent energy
eclbin: linear mesh for the solute-solvent energy
ecfbin: fine linear mesh for the solute-solvent energy
ec0bin: very fine linear mesh for the solute-solvent energy near 0
The default setting can be
eclbin = 5.0e-2 ; ecfbin = 2.0e-3 ; ec0bin = 2.0e-4 ; finfac = 10.0e0
ecdmin = -30.0e0 ; ecfmns = -0.20e0 ; ecdcen = 0.0e0 ; eccore = 20.0e0
ecdmax = 1.0e12 ; pecore = 300
See 6) of this document

The unit is kcal/mol

With the parameter setting above, the compile-option is given below

A) To read MARBLE trajectory

A1) on the 64-bit machine

```
$ mpiifort -O3 -ipo -fpic -shared-intel -mcmodel=large -autodouble
    -extend-source -cpp -Dtrjctry -DMARBLE –DMKL
    /opt/intel/mkl/10.2.2.025/include/mkl_dfti.f90 enganal.f
    -lmkl_intel_lp64 -lmkl_intel_thread -lmkl_core –lguide
$ rm -f *.o *.mod
```

And move a.out to the working directory

A2) on the 32-bit machine

To be written

B) To read GROMACS trajectory

B1) on the 64-bit machine

```
$ mpiifort -O3 -ipo -fpic -shared-intel -mcmodel=large -autodouble
    -extend-source -cpp -Dtrjctry -DGROMACS -DMKL
    /opt/intel/mkl/10.2.2.025/include/mkl_dfti.f90 enganal.f
    -lmkl_intel_lp64 -lmkl_intel_thread -lmkl_core –lguide
$ rm -f *.o *.mod
```

And move a.out to the working directory

B2) on the 32-bit machine

```
$ mpiifort -O3 -axW -ipo -autodouble -extend-source
    -cpp -Dtrjctry -DGROMACS –DMKL
    /opt/intel/mkl/10.0.1.014/include/mkl_dfti.f90 enganal.f
    -lmkl_lapack -lmkl_ia32 -lmkl_core -lguide
$ rm -f *.o *.mod
```

And move a.out to the working directory

C) To read NAMD trajectory

B1) on the 64-bit machine

```
$ mpiifort -O3 -ipo -fpic -shared-intel -mcmodel=large –autodouble
    -extend-source -cpp -Dtrjctry -DNAMD -DMKL
    /opt/intel/mkl/10.2.2.025/include/mkl_dfti.f90 enganal.f
```

```
        -lmkl_intel_lp64 -lmkl_intel_thread -lmkl_core –lguide
$ rm -f *.o *.mod
```

And move a.out to the working directory

B2) on the 32-bit machine

```
$ mpiifort -O3 -axW -ipo -autodouble -extend-source
      -cpp -Dtrjctry -DNAMD –DMKL
      /opt/intel/mkl/10.0.1.014/include/mkl_dfti.f90 enganal.f
      -lmkl_lapack -lmkl_ia32 -lmkl_core -lguide
$ rm -f *.o *.mod
```

And move a.out to the working directory

To calculate the solvation free energy from the energy distribution functions, slvfe.f is used. To compile slvfe.f the 64-bit machine

```
$ ifort -O3 -ipo -fpic -shared-intel -mcmodel=large -autodouble -extend-source
        slvfe.f -lmkl_intel_lp64 -lmkl_intel_thread -lmkl_core –lguide
```

To compile slvfe.f on the 32-bit machine

```
$ ifort -O3 -static -axW -ipo -autodouble -extend-source slvfe.f
        -lmkl_lapack -lmkl_ia32 -lmkl_core -lguide –lpthread
```

The program slvfe.f has 3 options to use.

A) basic

The most basic option of the program

The input and output looks as

```
$ ./a.out
   What is the distribution in solution?
"soln/engsln.tt"
   What is the energy density for insertion?
"refs/ engref.tt"
   What is the energy correlation for insertion?
"refs/ corref.tt"
   How many data are grouped into one?
1
   How many large-energy meshes are merged ? (in %)
0
```

What is the temperature of the system? (Kelvin)

300

What is average solute-solvent energy in solution?

-99.44138

Number of the 1-th solvent = 8878

Total solvation energy = -99.4413 kcal/mo

Total solvation free energy = -24.5777 kcal/mol

The first three are the distribution functions $\rho^e(\varepsilon)$, $\rho^e{}_0(\varepsilon)$, and $n^{(2)}(\varepsilon,\eta)$, respectively.

The 4th and 5th inputs are described in the Appendix of my part II paper; they correspond to the procedures for Eqs. (A5) and (A6), respectively. The 4th one coarsens the energy meshes, and the 5th is to validate (or invalidate) the choice of the maximum energy mesh. If the minimum energy mesh is larger than the actual minimum value of the solute-solvent interaction energy, the program halts.

The 7th input is the temperature in Kelvin.

The 8th temperature is the average sum of the solute-solvent interaction in solution obtained by MD. If the ermod program is used, it is in aveuv.tt

To activate the option "basic", open the slvfe.f program and set clcond = 'basic'

To read the energy-mesh information from an outside file, open the slvfe.f program and set peread = 'yes'

To obtain the average sum of the solute-solvent interaction energy from the distribution function $\rho^e(\varepsilon)$ in solution, open the slvfe.f program and set uvread = 'not'. In this case, the average sum is calculated as

$$\left(\text{average sum of the solute-solvent interaction energy}\right) = \sum_i \varepsilon_i n^e(i)$$

from the discretized form $n^e(i)$ of $\rho^e(\varepsilon)$ in solution and the 8th input is skipped.

To add the self-interaction energy after the calculation is done, set slfslt = 'yes'.

B) range

Extended option with automatic examination of the mesh error.

The compile option is the same as the above

To activate the option "basic", open the slvfe.f program and set clcond = 'range'

The variable corresponding to the 4th and 5th inputs of "basic" option are changed

internally in the program. The integer parameter numprm determines how many variations are conducted, and the variable settings (group and inft) are described within the loop 1001 of the subroutine defcond in the program slvfe.f. To change variable settings, open the slvfe.f and modify numprm and the loop 1001 of the subroutine defcond.

The temperature is set in the program as

```
real, parameter :: inptemp=300.0e0                    ! Kelvin
```

To change it, open the program and change the corresponding line.

To read the energy-mesh information from an outside file, open the slvfe.f program and set peread = ‘yes’. The same procedure as for the option “basic”

To obtain the average sum of the solute-solvent interaction energy from the distribution function $\rho^e(\varepsilon)$ in solution, open the slvfe.f program and set uvread = ‘not’. The same procedure as for the option “basic”

When uvread = ‘yes’, the input and output looks as

```
$ ./a.out
   What is the distribution in solution?
"soln/engsln.tt"
   What is the energy density for insertion?
"refs/engref.tt"
   What is the energy correlation for insertion?
"refs/corref.tt"
    What is average solute-solvent energy in solution?
-99.44138

   Number of the      1-th solvent    =              8878

   Total solvation energy          =          -99.44138    kcal/mol

                          chemical potential          difference
 1          0             -24.57774                    0.03100
 1         60             -24.57736                    0.03138
 1         80             -24.79076                   -0.18702
 1        100             -25.65188                   -0.95414
 2          0             -24.54011                    0.00663
 3          0             -24.60874                    0.00000
 4          0             -24.51221                    0.03454
```

5	0	-24.60105	-0.05531
5	60	-24.60765	-0.06090
5	80	-24.86456	-0.31782
5	100	-25.67749	-0.98775
8	0	-24.63969	-0.09395
10	0	-24.58602	-0.03928
15	0	-24.60326	-0.05652
20	0	-23.63020	0.91654

The 3 input functions correspond to the distribution functions $\rho^e(\varepsilon)$, $\rho^e{}_0(\varepsilon)$, and $n^{(2)}(\varepsilon,\eta)$, respectively.

C) merge

Most extended option with automatic examination of the mesh error and treatment of multiple files

The compile option is the same as the above

To activate the option "merge", open the slvfe.f program and set clcond = 'merge'

This option examines both the mesh error and statistical error

$\rho^e(\varepsilon)$ needs to be in soln directory and be named as

engsln.01, engsln.02, …engsln.*n*

$\rho^e{}_0(\varepsilon)$ and $n^{(2)}(\varepsilon,\eta)$ needs to be in refs directory and be named as

engref.01, engref.02, …engref.*n*, corref.01, corref.02, …corref.*n*

No input file is necessary and all the parameters need to be set in the file before compilation. After compilation. just $./a.out is enough.

numprm and variable settings (group and inft) to examine the mesh error are the same as those for the option 'range'

For temperature, peread, and uvread, the procedure is same as the option 'basic' and 'range'

If uvread = 'yes', aveuv.tt file needs to be prepared in the soln directory to read the average sum of the solute solvent interaction (there needs to be soln/aveuv.tt)

If slfslt = 'yes' to add the self-interaction energy after the calculation is done, weight_refs file needs to be prepared in the refs directory.

In the program, there is the line

```
integer, parameter :: numsln=20,numref=10,numdiv=numsln
```

This describes the number of files treated.

numsln is the number of files (number of blocks) for $\rho^e(\varepsilon)$
numref is the number of files (number of blocks) for $\rho^e{}_0(\varepsilon)$ and $n^{(2)}(\varepsilon,\eta)$.
numsln and numsln need not be the same
numdiv is the number of division. How it works is described within the subroutine datread in the program slvfe.f

The typical output reads

Number of the 1-th solvent = 1000

group	inft	solvation free energy	error	difference
1	0	-4.83682	0.29813	0.00119
1	60	-4.86093	0.29837	-0.02529
1	80	-4.99893	0.29939	-0.16330
1	100	-5.42438	0.30211	-0.58875
2	0	-4.84332	0.29850	-0.00531
3	0	-4.83801	0.30023	0.00000

….. (here, group and inft are the 4th and 5th inputs for the case of A) basic and are changed automatically as is so for the case of B) range

group	inft	Estimated free energy (kcal/mol)				
1	0	-4.7655	-4.5014	-5.1105	-4.3467	-4.9757
		-5.1904	-5.2384	-5.4530	-4.8877	-3.8990
1	60	-4.7892	-4.5243	-5.1346	-4.3718	-4.9996
		-5.2141	-5.2608	-5.4785	-4.9149	-3.9216
1	80	-4.9255	-4.6572	-5.2730	-4.5146	-5.1373
		-5.3498	-5.3946	-5.6220	-5.0618	-4.0535
1	100	-5.3476	-5.0701	-5.6996	-4.9506	-5.5626
		-5.7700	-5.8216	-6.0598	-5.4951	-4.4669
2	0	-4.7684	-4.5096	-5.1302	-4.3645	-4.9904
		-5.1897	-5.2402	-5.4654	-4.8745	-3.9004
3	0	-4.7639	-4.5084	-5.1220	-4.3527	-4.9865
		-5.1886	-5.2375	-5.4631	-4.8689	-3.8886

….. (here, at each set group and inft, the free-energy value is listed for each division. numdiv is 10 in this example, and there are 10 values for each

group and inft)

cumulative average & 95% error

1	-4.7639	
2	-4.6361	0.2555
3	-4.7981	0.3559
4	-4.6867	0.3361
5	-4.7467	0.2866
6	-4.8203	0.2765
7	-4.8799	0.2623
8	-4.9528	0.2699
9	-4.9435	0.2388
10	-4.8380	0.3002

(This is the cumulative average and correponsing 95% error along the divisions at group = 3 and inft = 0)

8) semi-automatic flow of calculations